



TECHNISCHE
UNIVERSITÄT
WIEN

MASTER'S THESIS

Development of a Framework for System Simulation

In fulfillment of the requirements for the academic degree

Master of Science

as part of the study program

Computational Science and Engineering

submitted by

Florian Frech, BSc

carried out at the Institute of Lightweight Design and Structural Biomechanics
at the TU Wien

under supervision of

Univ.Prof. Dipl.-Ing. Dr.techn. Dieter Pahr

and with collaboration of

Univ.Lektor Dipl.-Ing. Dr.techn. Harald Sima

March 24, 2026

Abstract

An abstract should concisely summarize your research by first stating the motivation or problem your work addresses, followed by the specific objectives or research questions you aimed to answer.

Next, briefly describe the key methods or approaches you employed, then present the most significant results or findings obtained. Finally, conclude by highlighting the main conclusions and the broader significance or implications of your research. Ensure the abstract is self-contained, clear, concise (typically 150-300 words), and includes relevant keywords.

Acknowledgements

Express your sincere and personal gratitude to everyone who supported you during your Master's thesis, even those not formally involved. Start with your supervisor and, if applicable, your co-supervisor, then mention family and friends for their support and encouragement. Briefly acknowledge specific help, such as proofreading or discussions. Keep it concise and write from the heart.

Typical Phrases:

Supervisor/Co-supervisor:

- "A special thank you goes to my supervisor, Prof. [Name], for their excellent guidance, valuable advice, and constructive discussions throughout this work."
- "I would like to sincerely thank my co-supervisor, Dr. [Name], for their dedicated support and helpful suggestions."

Family and Friends:

- "To my family, thank you from the bottom of my heart for your endless patience, support, and encouragement during this intense period."
- "A big thank you to my friends who always motivated and supported me."
- "I would like to thank [Name] for proofreading and their helpful comments."
- "Thanks to [Name] for the stimulating conversations and mental support."

Further Support:

- "I thank [Person/Institution] for providing [resources/data]."
- "Many thanks to [Person] for their technical assistance."

Concluding Remarks:

- "This thesis would not have been possible without your support. Thank you!"

Acronyms

Abstract

An abstract should concisely summarize your research by first stating the motivation or problem your work addresses, followed by the specific objectives or research questions you aimed to answer.

Next, briefly describe the key methods or approaches you employed, then present the most significant results or findings obtained. Finally, conclude by highlighting the main conclusions and the broader significance or implications of your research. Ensure the abstract is self-contained, clear, concise (typically 150-300 words), and includes relevant keywords.

Kurzfassung

Deine Kurzfassung fasst deine Forschungsarbeit prägnant zusammen, indem sie zuerst die Motivation oder das Problem deiner Arbeit darlegt, gefolgt von den spezifischen Zielen oder Forschungsfragen, die du beantworten wolltest.

Beschreibe anschließend kurz die wichtigsten Methoden oder Ansätze, die du verwendet hast, und präsentiere dann die wichtigsten Ergebnisse oder Erkenntnisse. Abschließend fasst du die wichtigsten Schlussfolgerungen und die breitere Bedeutung oder Implikationen deiner Forschung zusammen. Stelle sicher, dass die Kurzfassung in sich geschlossen, klar, prägnant (in der Regel 100-200 Wörter, prüfe spezifische Richtlinien) ist und relevante Schlüsselwörter enthält.

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	State of the Art	2
1.2.1	Equation-based Modeling and FMI-based Components	2
1.2.2	Domain-Specific Simulation Environments	3
1.2.3	Co-Simulation Frameworks and Orchestration	4
1.2.4	Hybrid Co-Simulation and Event Handling	5
1.2.5	Multi-Model and Multi-Fidelity Simulation	6
1.3	Problem Statement and Research Gap	7
1.4	Research Questions	7
1.5	Objectives	8
1.6	Thesis Outline	9
2	Theoretical Background	10
2.1	Continuous-Time System Modeling	10
2.1.1	Dynamical Systems with Inputs and Outputs	10
2.1.2	Ordinary and Differential-Algebraic Equations	11
2.1.3	Direct Feedthrough and Interface Variables	12
2.2	Hybrid System Modeling	13
2.2.1	Hybrid System Representation	13
2.2.2	Events, Zero-Crossings, and State Re-Initialization	14
2.2.3	Time Semantics in Hybrid Simulation	15
2.3	Co-Simulation Principles	15
2.3.1	Terminology	15
2.3.2	Functional Mock-up Interface and Functional Mock-up Units	16
2.3.3	Orchestrated Co-Simulation and Communication Steps	16
2.3.4	Dependency Analysis and Execution Order	16
2.3.5	Algebraic Loops in Coupled Systems	16
2.3.6	Hybrid Co-Simulation and Event Handling	16
2.4	Domain-Specific Modelling Foundations	16
2.4.1	Equation-Based Modelling with Modelica	16

2.4.2	Musculoskeletal Modelling with OpenSim	16
2.4.3	Finite Element Analysis with NGSolve	16
3	Requirements and Tool Selection	17
3.1	Requirements	17
3.1.1	User Requirements	17
3.1.2	System Requirements	19
3.2	Tool Comparison and Selection	20
3.2.1	Equation-Based Modeling	20
3.2.2	Musculoskeletal Modeling	21
3.2.3	Finite Element Modeling	22
3.2.4	Summary	23
4	Framework Architecture	24
4.1	Architectural Overview	24
4.2	Core Abstractions	27
4.2.1	CoSimComponent Interface	27
4.2.2	System and Connections	28
4.2.3	Algorithm Interface	29
4.2.4	Multi-Model Component	30
4.3	Summary	31
5	Feature Implementation and Verification	32
5.1	Port System and Unit-Aware Connections	32
5.1.1	Port Specification and Port State	33
5.1.2	Physical Unit Handling	34
5.1.3	Connection Validation	35
5.1.4	Verification	35
5.2	Component Wrappers	36
5.2.1	FMI 2.0 Co-Simulation Wrapper	36
5.2.2	Musculoskeletal Model Wrapper	36
5.2.3	Finite Element Model Wrapper	36
5.3	Structural Analysis and Execution Ordering	36
5.4	Algebraic Loop Resolution	36
5.5	Master Algorithms	36
5.6	Hybrid Co-Simulation Algorithm	36
5.7	Multi-Model Component and Mode Switching	36
5.8	System Graph Visualization	36
6	Case Study: Controlled Pendulum System	37
6.1	Choice and Justification of the Case Study	37
6.2	System Engineering of the Controlled Pendulum	37
6.2.1	System Architecture and Signal Flow	37

6.2.2	PID Controller	37
6.2.3	BLDC Drive Model	37
6.2.4	Sensors and Quantization	37
6.3	Pendulum Models	37
6.3.1	Modelica Pendulum	37
6.3.2	OpenSim Pendulum	37
6.3.3	FEM Pendulum	37
6.3.4	Multi-Model Pendulum	38
6.4	System Assembly and Configuration	38
6.4.1	Component Wiring and Connection Graph	38
6.4.2	Algebraic Loop Analysis	38
6.4.3	Monolithic Modelica Reference Model	38
7	Discussion and Conclusion	39
7.1	Discussion of Results	39
7.2	Limitations	39
7.3	Conclusions	39
7.4	Outlook and Future Work	39
	List of Figures	40
	List of Tables	41
	List of Listings	42
	Bibliography	43
.5	Complete Derived System Requirement Specification	46
.5.1	Modeling	46
.5.2	Heterogeneity	48
.5.3	Simulation	48
.5.4	Interaction	49

Introduction

1.1 Context and Motivation

Active mechatronic devices such as powered prostheses and exoskeletons consist of multiple interacting subsystems. These include mechanical structures, actuators, sensors, and embedded controllers. Together, they form tightly coupled multi-domain systems. Such systems are typical cyber-physical systems because computation and physical behavior are linked through sensing, actuation, and feedback [1]. A change in one subsystem can influence the behavior of the whole device. For example, modifications in the control law, actuator design, or geometry of the mechanical structure can affect loads, stability, performance, and safety. For this reason, system-level simulation is needed already in early design phases, before physical prototypes are available [2].

For devices that interact with the human body, this challenge becomes even more demanding. A prosthetic knee or an active exoskeleton is not merely a technical device. Its function depends on the coupled dynamics of the device and the musculoskeletal system of the user. In addition, local structural effects such as compliance, deformation, and contact may become relevant in critical operating phases. Questions such as whether a controller remains stable during ground contact, how loads are transferred through compliant components, or how device forces interact with biological tissue cannot be answered from isolated component analyses alone. They require virtual testing environments that represent both the technical system and its biomechanical context in a single, time-synchronous simulation [3].

In engineering practice, however, the required subsystem models are rarely created within a single tool. Mechanical and structural behavior is often derived from geometry-based models in computer-aided design and finite element analysis environments. Control-oriented, electrical, and other system-level subsystems are commonly formulated as one-dimensional equation-based models, for example in Modelica [4]. Musculoskeletal dynamics, in turn, are modeled in specialized biomechanics software such as OpenSim [5]. These modeling environments serve different purposes, operate at different abstraction levels, and use different numerical representations. Modern system development therefore requires not only reusable models within individual domains, but also a practical way to integrate heterogeneous models into a coherent system-level simulation workflow.

Co-simulation provides a natural integration strategy for such heterogeneous models. Each subsystem retains its domain-specific solver and model representation, while a master algorithm orchestrates data exchange at discrete communication points [6]. The Functional Mock-up Interface (FMI) standardizes the component interface for this purpose and is widely adopted in industry [7].

However, FMI defines the *interface* to individual simulation components, but it does not prescribe how multiple components are orchestrated into a consistent system simulation [6]. Several challenges remain when heterogeneous models are coupled in practice. These challenges include determining a correct execution order when component outputs depend algebraically on inputs [8], detecting and resolving circular dependencies between components [9], and handling discrete events such as mechanical contact or discrete mode changes during otherwise continuous operation [10–12].

Moreover, different representations of the same physical subsystem involve different trade-offs between computational cost and the level of detail provided by the simulation. A rigid-body model computes global kinematics and kinetics efficiently, but it does not explicitly resolve local deformation or stress. A finite element model can resolve these local field quantities, but at significantly higher computational cost. In system simulation, this suggests that detailed models should be activated selectively. For example, a finite element representation may only be needed during contact phases in which local deformation is relevant, whereas a computationally inexpensive rigid-body model may be sufficient during free motion. To enable such selective model use, the simulation framework must support switching between alternative models of the same subsystem at runtime, together with consistent state transfer across model boundaries [13].

1.2 State of the Art

This section positions the thesis within the current landscape of modeling and simulation approaches relevant to heterogeneous hybrid system simulation. It does not aim to provide an exhaustive review. Instead, it highlights the main modeling paradigms, tool classes, and orchestration challenges that form the context for the development of *SysSimX*.

1.2.1 Equation-based Modeling and FMI-based Components

Equation-based modeling describes systems through mathematical relations between physical quantities. In simulation, these relations are formulated as algebraic equations, differential equations, or combinations of both. Two main modeling styles can be distinguished. In causal modeling, the computational direction is defined explicitly through inputs and outputs. In acausal modeling, the relations between variables are stated as equations, and the simulation tool derives the required computational causality. Both approaches are relevant for cyber-physical and mechatronic systems. Causal models are common for control, signal processing, and software-oriented subsystems. Acausal equation-based models are especially useful for physical subsystems whose behavior follows conservation laws and constitutive

relations.

Equation-based modeling has become an established approach for the description of multi-domain physical systems. It allows engineers to formulate subsystem behavior in a compact and domain-oriented way without prescribing the full numerical solution procedure. This is particularly useful for mechatronic systems, since such systems combine mechanics, electronics, thermal effects, hydraulics, and control-oriented behavior. A common formalism helps to represent these domains within one model structure and to connect them through physically meaningful interfaces.

The most prominent open representative of this paradigm is Modelica [4]. Modelica is an object-oriented, equation-based language for the modeling of physical systems. It supports hierarchical composition, reusable component libraries, and acausal connectors. Through these features, it is widely used for the modeling of multi-domain systems in both academia and industry. Commercial and open-source environments such as Dymola, SimulationX, and OpenModelica provide solvers, graphical editors, and libraries based on the Modelica language and the Modelica Standard Library.

For tool interoperability, the Functional Mock-up Interface (FMI) standard has become particularly important [7]. It provides a standardized way to package and exchange simulation models across tools and therefore plays a central role in component-based simulation workflows [6].

Equation-based tools and FMI-based components provide a strong basis for system simulation. However, they do not cover all domain-specific modeling needs equally well.

1.2.2 Domain-Specific Simulation Environments

Although equation-based modeling environments cover many physical domains, they do not represent all simulation needs equally well. In particular, some subsystem classes require specialized modeling approaches and dedicated software tools. This is especially relevant for the systems considered in this thesis, where technical subsystems interact with the human musculoskeletal system and where local structural effects may become important.

A first example is musculoskeletal simulation. In applications such as prostheses and exoskeletons, the technical device is mechanically coupled to the human body [3]. The resulting system behavior depends not only on the device itself, but also on the biomechanics of the user. For such questions, musculoskeletal simulation environments provide modeling capabilities that go beyond classical one-dimensional component libraries. OpenSim is a widely used open platform in this area [5, 14]. It supports the modeling of articulated rigid-body systems with joints, muscle-tendon actuation, and biomechanical constraints. It also provides established workflows such as forward dynamics and inverse dynamics. In this way, OpenSim enables analyses that are highly relevant for human-device interaction, but it is not primarily designed as a standardized FMI-based component source.

A second example is finite element analysis. Finite element models are used when spatially distributed effects must be resolved explicitly. This includes deformation, stress, strain, and contact phenomena. Such information cannot usually be obtained from lumped-

parameter rigid-body models alone. Finite element methods solve partial differential equations on a spatial discretization of the geometry and therefore provide a much higher level of local detail [15]. This is important when compliant components, structural loading, or local contact behavior must be analyzed.

Musculoskeletal models and finite element models differ substantially from classical equation-based system components. Their interfaces are often not defined through a small set of reusable connectors. Instead, coupling variables must be chosen specifically for the application. These variables may include generalized coordinates, torques, external forces, contact quantities, marker trajectories, or reduced state representations. As a result, such models are usually integrated through custom wrappers or tool-specific interfaces rather than through direct FMI export.

For heterogeneous system simulation, this creates an important challenge. Domain-specific tools provide essential high-fidelity models, but they are not naturally embedded in the standardized component-oriented workflows that are common in FMI-based environments. A practical system simulation framework must therefore be able to integrate both standardized FMU-based models and non-FMI domain-specific models within one common orchestration concept.

1.2.3 Co-Simulation Frameworks and Orchestration

Co-simulation enables the coupled simulation of subsystem models that originate from different tools. In this setting, each subsystem keeps its own model representation and, in many cases, its own numerical solver. The coupled simulation is coordinated through data exchange at communication points. This makes co-simulation a natural approach for heterogeneous system simulation.

The most important standard in this area is the Functional Mock-up Interface (FMI) [7, 16]. FMI defines a standardized interface for simulation components called Functional Mock-up Units (FMUs). An importing environment creates FMU instances and calls their standardized interface functions. In the case of Model Exchange, the importing tool provides the numerical solver and performs the integration. In the case of Co-Simulation, the FMU typically advances its own internal solver and exchanges values only at communication points. FMI 3 further extends this concept through Scheduled Execution and explicit clock-based mechanisms for discrete-time behavior and event-oriented interaction [16].

Building on FMI, several frameworks have been developed to orchestrate coupled FMU execution. Standalone tools such as OMSimulator provide FMI-based composition and execution environments with scripting and graphical support [17]. INTO-CPS Maestro provides another orchestration environment for FMI-based co-simulation and supports the specification and execution of coupled simulation scenarios [18]. These tools show that FMI-based co-simulation is well established at the level of standardized component coupling.

Within the Python ecosystem, several libraries address FMI at different abstraction levels. FMPy provides lightweight support for loading, inspecting, validating, and simulating FMUs [19]. PyFMI offers a more simulation-oriented Python interface for Model Exchange and

Co-Simulation FMUs [20]. Research-oriented tools such as CoFmuPy further show that lightweight Python-based co-simulation environments can be built around FMI-compliant components [21]. Taken together, these tools show that FMI-based orchestration is mature for FMU-centric workflows. This is especially true when subsystem models are available through standardized component interfaces.

At the same time, the scope of most existing co-simulation frameworks remains strongly FMU-centered. Their direct interoperability is strongest when subsystem models are available as FMI-compliant components. Models from tools that do not naturally export FMUs, such as musculoskeletal simulation environments or finite element solvers, usually require custom wrappers or tool-specific interfaces. In addition, FMI standardizes the component interface, but it does not by itself solve all orchestration problems at system level. Questions such as execution order, algebraic-loop handling, hybrid event handling, and runtime switching between alternative subsystem models remain the responsibility of the importing framework. For this reason, current co-simulation frameworks provide an essential basis for interoperability. However, they do not fully address the broader requirements of heterogeneous hybrid system simulation.

1.2.4 Hybrid Co-Simulation and Event Handling

Hybrid systems combine continuous-time dynamics with discrete changes of state or behavior. In cyber-physical and mechatronic systems, such behavior arises in several ways. Typical examples are sampled sensors, digital controllers, switching logic, mode changes, and mechanical contact. Their simulation therefore requires both the numerical solution of the continuous-time problem and the consistent handling of discrete events.

In a monolithic simulation, one solver has access to the full system model. It can evaluate all relevant variables and equations during each integration step. This makes it possible to detect event conditions, localize the event time, execute the required event logic, and then continue the continuous integration from a consistent post-event state [4]. For hybrid systems, this is a major advantage of a monolithic formulation.

In co-simulation, the situation is more difficult. The master algorithm usually sees subsystem outputs only at communication points, while each component advances internally between these points. As a result, an event may occur within a communication step without being immediately visible at system level. Hybrid co-simulation therefore requires additional orchestration mechanisms. The importing framework must be able to detect that an event has occurred, localize the event time, restore or roll back subsystem states when necessary, execute the event handling logic, synchronize the affected components, and only then resume the continuous simulation [10]. The problem becomes even more difficult when several events interact or when one event triggers further events in other subsystems.

This challenge has been discussed extensively in the co-simulation literature. In particular, the representation of time, event handling, and rollback capabilities have been identified as central issues for deterministic and robust hybrid co-simulation [8, 11, 12]. FMI 3 extends FMI 2 with additional concepts that are relevant in this context, including clocks,

Event Mode, and Scheduled Execution [16]. These additions improve the representation of sampled and event-driven behavior. However, the standard alone does not solve the full orchestration problem. The importing framework must still implement the system-level logic for event detection, event localization, state consistency, and continuation after events.

Hybrid co-simulation is therefore a central topic for the present thesis. The systems considered here include both sampled and event-driven behavior, for example sensor sampling, controller action, and mechanical contact. A heterogeneous simulation framework must be able to combine continuous integration with robust event handling across coupled subsystems.

1.2.5 Multi-Model and Multi-Fidelity Simulation

Many engineering subsystems can be represented by more than one simulation model. These models may differ in their physical assumptions, numerical complexity, computational cost, and output detail. Model fidelity denotes how closely a model represents the system properties of interest [22]. This leads to the idea of multi-model or multi-fidelity simulation [23, 24].

A low-dimensional or reduced-complexity model is often sufficient when the objective is to predict global system behavior, design controllers, or perform rapid parameter studies. More detailed models are needed when local effects become important, for example deformation, stress, or contact. The appropriate model therefore depends on the simulation objective and on the current operating phase of the system [13, 25].

In engineering practice, this trade-off appears both across the development process and during runtime. Early design stages often rely on simple models because they are fast to build and simulate. Later stages require more detailed models for validation. A similar principle can also be used within one simulation scenario, where different phases of motion require different levels of detail.

Region-based runtime model substitution has been proposed as a strategy for simulation speed-up. Choi et al. describe a multi-fidelity methodology in which a computationally expensive target model is replaced by lower-fidelity models outside an interest region while preserving a common external interface and switching between internal models according to fidelity-change conditions [13, 26]. The present thesis is conceptually related to this idea, but applies it in a narrower sense. The framework switches between heterogeneous subsystem models from different tools that share a common external interface, rather than between simplified derivatives of one target model.

Existing work on multi-fidelity simulation covers a broad range of approaches, including hierarchical model families, reduced-order models, surrogate models, and adaptive model selection [23, 24]. For the present thesis, the most relevant aspect is the practical switching between alternative subsystem models within one simulation workflow. In this setting, the main challenge is not only the existence of models with different fidelity, but also the consistent transition between them. The framework must preserve a common external interface, transfer the relevant state information, define suitable switching criteria, and avoid unstable

or excessively frequent mode changes [25]. In heterogeneous settings, such switching typically requires projection onto a shared reduced state rather than exact preservation of all internal model variables.

1.3 Problem Statement and Research Gap

Equation-based simulation, musculoskeletal simulation, finite element analysis, and FMI-based co-simulation are well established within their respective domains. However, many mechatronic and biomechanical systems require these modeling paradigms to be combined within one system-level simulation workflow. In practice, such integration remains difficult. Existing approaches are strong either in standardized FMU-based coupling or in domain-specific analysis, but they provide only limited support for the unified integration of FMU models, musculoskeletal models, and finite-element models within one common simulation environment.

This limitation becomes particularly relevant when heterogeneous tool coupling must also be combined with dependency-aware execution, algebraic-loop handling, hybrid event handling, and runtime switching between alternative subsystem models. In this area, a practical and reproducible workflow is still insufficiently addressed. As a result, the analysis of systems whose relevant physics and appropriate model representation change across operating phases remains inconvenient, for example when transitioning between free motion, human interaction, and contact-dominated behavior.

This thesis addresses this methodological gap through the development and evaluation of the *SysSimX* framework for heterogeneous hybrid system simulation. More specifically, the thesis makes the following contributions:

- the definition of a unified component-based abstraction for heterogeneous subsystem models,
- the development of a graph-based orchestration approach with dependency analysis, execution-order resolution, and algebraic-loop handling,
- the integration of hybrid event handling within a common co-simulation workflow,
- the introduction of a multi-model component concept for runtime switching between alternative subsystem models with consistent state synchronization, and
- the implementation and evaluation of a controlled-pendulum case study combining FMU, OpenSim, and FEM models.

1.4 Research Questions

This thesis investigates how heterogeneous subsystem models from different simulation environments can be integrated into a coherent system-level simulation workflow. The central research question is formulated as follows:

How can a framework support heterogeneous hybrid co-simulation of subsystem models from different tools while enabling consistent orchestration, event handling, and runtime switching between alternative models of the same subsystem?

To answer this main question, the following sub-questions are addressed:

- RQ1** How can heterogeneous simulation models from equation-based, musculoskeletal, and finite element tools be represented through a unified component interface for co-simulation?
- RQ2** How can dependencies between heterogeneous components be analyzed in order to derive a consistent execution order and to identify direct feedthrough and algebraic loops?
- RQ3** How can hybrid phenomena, such as contact, switching, and discrete state changes, be detected, localized, and handled within the same co-simulation workflow?
- RQ4** How can runtime switching between alternative subsystem models, including low-dimensional and more detailed representations, be achieved with sufficiently consistent state transfer?
- RQ5** To what extent can the resulting framework reproduce relevant system behavior and support representative benchmark scenarios such as the controlled pendulum?

1.5 Objectives

Based on these research questions, the overall objective of this thesis is to develop and evaluate the *SysSimX* framework as a practical approach for heterogeneous hybrid system simulation. In particular, the thesis pursues the following objectives:

- O1** Design a unified component-based abstraction for heterogeneous subsystem models.
- O2** Develop a graph-based orchestration concept for coupled system models, including dependency analysis and execution-order resolution.
- O3** Implement support for direct feedthrough analysis and algebraic-loop handling within the orchestration workflow.
- O4** Integrate hybrid event handling, including event detection, event localization, and event execution, into the framework.
- O5** Develop a multi-model component concept that enables runtime switching between alternative subsystem models with consistent state synchronization.
- O6** Apply the framework to a controlled-pendulum case study combining FMU, OpenSim, and FEM models.
- O7** Evaluate the framework with respect to functional correctness, consistency of coupled behavior, and practical applicability in representative simulation scenarios.

1.6 Thesis Outline

Theoretical Background

Purpose: Only the theory that is prerequisite to understanding multiple features. Written to be read once, front-to-back.

2.1 Continuous-Time System Modeling

2.1.1 Dynamical Systems with Inputs and Outputs

A model is a mathematical representation of a real system. It is used to describe the system behavior and to answer questions by analysis or simulation. The same physical system can be represented by different models. The choice of model depends on the question of interest and on the required level of detail [27, 28].

A dynamical system is a system whose behavior changes over time. In such systems, the effect of an action does not occur instantaneously. Instead, the future behavior depends on the current condition of the system and on external influences. Examples are mechanical motion, electrical circuits, and thermal processes [27].

In engineering, complex systems are usually described as compositions of smaller subsystems or components. Each component interacts with its environment through inputs and outputs. Inputs represent external influences acting on the component. Outputs represent variables that are provided to other components or to the environment. This input-output view is useful because it supports the modular description of larger systems from connected subsystems [28].

In addition to inputs and outputs, a dynamical system is characterized by its state. The state is a set of variables that contains the information required to predict the future evolution of the system for a given input. The set of all possible states is called the state space. For physical systems, state variables often represent quantities such as position, velocity, momentum, or stored energy [27, 28].

A common representation of continuous-time dynamical systems is the state-space form

$$\dot{x} = f(x, u, t), \quad y = h(x, u, t), \quad (2.1)$$

where x is the state vector, u is the input vector, and y is the output vector. The function f describes the time evolution of the state. The function h defines how the outputs are

obtained from the state and the inputs. If the functions do not depend explicitly on time, the system is called time-invariant [27].

As a simple example, consider an RC circuit with input voltage $u(t)$ and capacitor voltage $v_C(t)$. The capacitor voltage stores the past effect of the input and is therefore a natural state variable. With

$$x(t) = v_C(t),$$

the system can be written as

$$\dot{x} = -\frac{1}{RC}x + \frac{1}{RC}u, \quad y = x. \quad (2.2)$$

This is a continuous-time system with one state, one input, and one output. Its future behavior depends on the current capacitor voltage and on the applied input voltage.

For linear time-invariant systems, the state-space form reduces to

$$\dot{x} = Ax + Bu, \quad y = Cx + Du, \quad (2.3)$$

where A , B , C , and D are constant matrices. This representation is widely used in systems and control theory. It also makes clear that the output may depend not only on the state, but also directly on the input through the matrix D . This property is important in co-simulation, because it is closely related to the concept of direct feedthrough discussed later in this thesis [27].

2.1.2 Ordinary and Differential-Algebraic Equations

Many continuous-time systems can be described by ordinary differential equations. In this case, the time derivative of the state is given explicitly as a function of the current state, the input, and time [29, 30]. A general nonlinear ODE can be written as

$$\dot{x} = f(x, u, t), \quad (2.4)$$

where x is the state vector and u is the input vector.

However, not all systems can be written naturally in explicit ODE form. In many physical and component-based models, additional algebraic constraints are present. These constraints may arise from constitutive relations, conservation laws, interface conditions, or the interconnection of subsystems [28, 30]. In such cases, the model is more naturally written as a differential-algebraic equation.

A general DAE can be written in implicit form as

$$F(t, z, \dot{z}, u) = 0, \quad (2.5)$$

where z contains the model variables and u denotes the input. In contrast to an ODE, the derivative is not given explicitly. Instead, differential and algebraic relations are stated together [29].

A common semi-explicit form of a DAE is

$$\dot{x} = f(x, w, u, t), \quad (2.6)$$

$$0 = g(x, w, u, t), \quad (2.7)$$

where x contains the differential states and w contains algebraic variables. The first equation describes the time evolution of the states. The second equation imposes algebraic constraints that must hold at every time instant. This form is useful because it separates dynamic behavior from purely algebraic relations [29, 30].

In contrast to ODEs, a DAE does not in general admit arbitrary initial values. The initial state must be consistent with the algebraic constraints. In addition, DAEs are often classified by an index that reflects the structural difficulty of the problem. Higher-index systems are generally harder to analyse and simulate [29].

DAEs are especially important in component-based and equation-based modelling. When subsystems are connected, additional interface equations are introduced. As a result, variables that appear to be independent in isolated subsystem models may become constrained in the coupled system. For this reason, DAE formulations are often more suitable than pure ODE formulations for multi-domain physical systems [28, 30].

This is particularly relevant for modern modeling and simulation workflows. Subsystem models are often created separately and then coupled through algebraic constraints. In such settings, it is usually preferable to treat the DAE directly rather than to transform it manually into an ODE [29].

This distinction is relevant for the present thesis. Equation-based models exported as FMUs may contain direct dependencies and algebraic relations at their interfaces. These relations influence execution ordering, direct feedthrough analysis, and the treatment of algebraic loops in co-simulation. For this reason, ODE and DAE models form the mathematical basis for the system representations considered in *SysSimX*.

2.1.3 Direct Feedthrough and Interface Variables

When a system is decomposed into subsystems, each subsystem interacts with its environment through interface variables. These variables describe the quantities that cross the subsystem boundary. In a general systems view, they are represented as inputs and outputs. In physical systems, they may correspond to quantities such as force and velocity, voltage and current, or more abstract control and measurement signals [27, 28].

The input-output view is useful because it allows complex systems to be described as networks of connected subsystems. Each subsystem can then be studied and implemented as a separate model, while the overall behavior is obtained from the interconnection of these models [27, 28]. This is a central idea in component-based modeling and simulation.

Consider a subsystem in state-space form

$$\dot{x} = f(x, u, t), \quad y = h(x, u, t), \quad (2.8)$$

where x is the state vector, u is the input vector, and y is the output vector. A subsystem has direct feedthrough if the current output depends directly on the current input. If the output depends only on the state, then no direct feedthrough is present. In the linear time-invariant case,

$$\dot{x} = Ax + Bu, \quad y = Cx + Du, \quad (2.9)$$

direct feedthrough is present if the matrix D is nonzero [27].

This distinction is important in interconnected systems. If the output of one subsystem depends instantaneously on its input, and this output is connected to the input of another subsystem, a zero-delay dependency is created. If such dependencies form a closed loop, the coupled system may no longer be representable as a pure ODE. Instead, algebraic equations must be solved together with the differential equations, which leads naturally to a DAE or an algebraic loop [27, 29, 30].

In component-based physical modeling, interface variables are therefore not only communication signals. They also define how subsystem equations are coupled across their boundaries. This is particularly relevant in equation-based and multi-domain models, where interface conditions may introduce algebraic constraints between connected subsystems [28, 30].

For the present thesis, direct feedthrough is an important structural property of simulation components. It determines whether current outputs can be evaluated independently of current inputs or whether instantaneous dependencies must be taken into account. This distinction is used later to derive execution order, identify zero-delay dependencies, and detect algebraic loops in coupled co-simulation systems.

2.2 Hybrid System Modeling

2.2.1 Hybrid System Representation

Cyber-physical systems combine continuous physical processes with digital control and software. Further, impacts, switching logic, and sampled controllers can cause abrupt changes in modes, derivatives, or states. These changes and transitions happen in a much faster way than the usual changes in the system and are therefore considered as instantaneous events. A purely continuous model cannot describe such behavior in a consistent way. Hybrid models therefore combine continuous dynamics with discrete change [31, 32].

In this thesis, we use the reduced hybrid representation

$$(q(t), x(t), z(t), u(t), y(t)), \quad (2.10)$$

where $q(t)$ is the discrete state, $x(t)$ is the continuous state, $z(t)$ collects algebraic variables, $u(t)$ is the input, and $y(t)$ is the output. The discrete state q may represent an operating mode, a Boolean logic state, or an internal controller state. This notation is a reduced representation inspired by formal hybrid modeling approaches such as Omola [33, 34].

Between events, the discrete state is constant and the continuous part follows the active

continuous-time model:

$$0 = F_{q(t)}(\dot{x}(t), x(t), z(t), u(t), t), \quad y(t) = h_{q(t)}(x(t), z(t), u(t), t), \quad t \neq t_e. \quad (2.11)$$

At an event time t_e , the model may change its discrete state and may reset selected continuous states:

$$x^+ = R_j(x^-, z^-, q^-, u^-, t_e), \quad q^+ = \delta_j(x^-, z^-, q^-, u^-, t_e). \quad (2.12)$$

The function $R_j(\cdot)$ computes the post-event values of the continuous states. The function $\delta_j(\cdot)$ updates the discrete state. Here, x^- and q^- denote the values immediately before the event, and x^+ and q^+ denote the values immediately after the event. The system trajectory is therefore piecewise continuous [35, 36].

Two event effects are important in this thesis. A switching event changes the active mode or equations, but the continuous state remains continuous. A reset event changes one or more continuous states instantaneously. The first changes the derivative, while the second produces a jump in the state trajectory [36].

A simple example is an integrator with reset:

$$\dot{x}(t) = 1, \quad y(t) = x(t). \quad (2.13)$$

When the state reaches a threshold, an event is triggered and the state is reset:

$$x \geq 1 \quad \Rightarrow \quad x^+ = 0. \quad (2.14)$$

The state evolves continuously until the threshold is reached. At the event time, the state jumps to zero. After the reset, continuous evolution starts again from the new state.

2.2.2 Events, Zero-Crossings, and State Re-Initialization

Hybrid models change their behavior at event times. Two event trigger types are common. Time events occur at known or scheduled instants. State events depend on the continuous trajectory and are therefore not known in advance [31, 36].

In this thesis, state events are represented by event indicators, also called zero-crossing functions,

$$g_j(x(t), z(t), q(t), u(t), t). \quad (2.15)$$

An event of type j is triggered when the indicator crosses zero. For example, the condition $x \geq x_{\max}$ can be written as

$$g(x) = x - x_{\max}. \quad (2.16)$$

The event occurs when $g(x) = 0$ [37].

In numerical simulation, the event time is usually not located exactly at the end of an integration step. The simulator first detects that a zero-crossing occurred within a time interval. It then localizes the event time inside this interval with a root-finding procedure. This avoids integrating across a discontinuity without resolving its time of occurrence [31,

33].

Once the event time t_e has been found, the update maps introduced in Section 2.2.1 are evaluated. The resulting post-event state must satisfy the active continuous and algebraic equations before simulation can continue. This step is called state re-initialization or restart [33, 38].

2.2.3 Time Semantics in Hybrid Simulation

Hybrid simulation alternates between phases of continuous evolution and discrete event handling. During continuous phases, the state evolves according to the active differential-algebraic equations. At event times, discrete updates may change the discrete state, reset continuous states, or trigger further events.

A purely real-valued notion of time is often not sufficient for this process. Several discrete updates may have to be executed at the same physical time. This occurs, for example, when one event triggers another event immediately, or when several event conditions become active at the same instant [38].

For this reason, hybrid simulation often uses *superdense time*. A time point is then represented by the pair

$$(t, n), \tag{2.17}$$

where $t \in \mathbb{R}_{\geq 0}$ denotes the physical time and $n \in \mathbb{N}_0$ denotes a logical micro-step index. The physical time identifies when the event occurs. The index n orders the sequence of discrete updates at that same physical time. An increase in n does not represent additional physical time. It only represents progress in the event iteration at fixed time t [10, 12].

Superdense time makes it possible to distinguish between the state before an event and the states after one or more discrete updates at the same physical time. Continuous integration resumes only after all event updates at the current physical time have been processed. This distinction is important for the present thesis, since the hybrid co-simulation algorithm introduced later first localizes an event in physical time and then performs ordered event handling at that same time instant.

2.3 Co-Simulation Principles

The following section contains the relevant background and state of the art regarding co-simulation. First we classify the concepts in the field of coupled modeling and simulation.

2.3.1 Terminology

It is important to

2.3.2 Functional Mock-up Interface and Functional Mock-up Units

2.3.3 Orchestrated Co-Simulation and Communication Steps

2.3.4 Dependency Analysis and Execution Order

2.3.5 Algebraic Loops in Coupled Systems

2.3.6 Hybrid Co-Simulation and Event Handling

2.4 Domain-Specific Modelling Foundations

2.4.1 Equation-Based Modelling with Modelica

Principles of Acausal, Equation-Based Modeling

Causal Component Interfaces

FMU Export

2.4.2 Musculoskeletal Modelling with OpenSim

Kinematic Chains

Forward Dynamics

Hunt-Crossles Contact

2.4.3 Finite Element Analysis with NGSolve

Deformable Body Dynamics

Newmark Integration

Torque Application

Contact Modeling using the Penalty Method

Requirements and Tool Selection

This chapter defines the requirements that guided the design of the *SysSimX* framework and the selection of the external tools used in this thesis. The requirements were elicited iteratively from stakeholder discussions, the target application scenario, and an analysis of existing modeling and simulation tools. First, user requirements were formulated to describe the expected capabilities of the overall heterogeneous hybrid system simulation workflow. Second, technical system requirements were derived from these user requirements in order to guide the architecture and implementation of *SysSimX*. Throughout this chapter, the requirements are formulated in a tool-agnostic way, while concrete tool choices are justified afterwards in the tool selection section.

3.1 Requirements

Requirement priorities are expressed by the terms *shall* and *should*. A *shall* requirement is considered mandatory within the scope of this thesis, whereas a *should* requirement denotes a desirable but non-essential capability.

3.1.1 User Requirements

In this thesis, the user requirements are represented primarily by functional requirements, since these directly motivate the architecture and implementation of *SysSimX*. For clarity, they are grouped into four categories: modeling, heterogeneity, simulation, and user interaction. Non-functional considerations such as usability, robustness, portability, and practical performance also influenced the design, but they are only treated briefly since they are not the primary focus of the functional implementation and evaluation.

Modeling

The modeling requirements define how users shall represent subsystems and assemble them into complete system models. They emphasize modular structure, explicit physical interfaces, and support for the main model classes required in this thesis, namely equation-based, musculoskeletal, and continuum-mechanical models.

ID	Requirement	Priority
UR_01	The overall workflow shall support component-based modeling of subsystems from different physical domains.	Shall
UR_02	The overall workflow shall support explicit units for physical quantities at model interfaces and parameters.	Shall
UR_03	The overall workflow shall support hierarchical composition of atomic components and virtual subsystems.	Shall
UR_04	The overall workflow shall support equation-based modeling of subsystem behavior.	Shall
UR_05	The overall workflow shall support musculoskeletal modeling of biomechanical subsystems.	Shall
UR_06	The overall workflow shall support continuum-mechanical modeling with spatial discretization for deformable subsystems.	Shall
UR_07	The overall workflow shall support subsystem models with continuous-time, discrete-time, and hybrid behavior.	Shall
UR_08	The overall workflow shall support reuse and parameterization of components and subsystems across different system configurations.	Shall

Heterogeneity

The heterogeneity requirements describe the need to combine subsystem models that originate from different modeling paradigms and execution environments. They define the interoperability scope of the workflow and motivate the need for a common co-simulation framework.

ID	Requirement	Priority
UR_09	The overall workflow shall support the integration of subsystem models developed in different modeling environments.	Shall
UR_10	The overall workflow shall support the combined use of equation-based, musculoskeletal, and continuum-mechanical subsystem models within one simulation setup.	Shall

Simulation

The simulation requirements describe the capabilities needed to execute the coupled system model. They focus on hybrid system behavior, master-level coordination, and the configuration of workflow-level simulation settings.

ID	Requirement	Priority
UR_11	The overall workflow shall enable the numerical simulation of heterogeneous hybrid systems with continuous and discrete behavior.	Shall
UR_12	The overall workflow shall allow users to select the co-simulation master algorithm, configure the macro step size, and provide tool-specific simulation settings where required.	Shall

User Interaction

The interaction requirements describe how users shall inspect the system structure, monitor simulations, and work with the resulting data. They focus on transparency, comparison of alternatives, and practical usability of the workflow.

ID	Requirement	Priority
UR_13	The overall workflow should provide a visual representation of the system model and its hierarchical structure.	Should
UR_14	The overall workflow should allow users to observe simulation progress through textual and graphical feedback.	Should
UR_15	The overall workflow should allow users to compare simulation results across model variants and configurations.	Should
UR_16	The overall workflow should support result export and post-processing for further analysis and documentation.	Should

3.1.2 System Requirements

The user requirements described above were translated into technical system requirements for the *SysSimX* framework. These derived system requirements specify the capabilities that the framework itself must provide in order to realize the intended heterogeneous hybrid system simulation workflow. Here, only a compact overview is given, the complete system requirement specification is provided in the appendix.

The *modeling* requirements define the common abstractions that are needed to represent subsystem models, their interfaces, and their composition into complete system models. This includes reusable component abstractions, explicit signal and event interfaces, parameterization, and unit-aware model definitions.

The *heterogeneity* requirements define the capabilities needed to integrate subsystem models from different modeling environments into one common workflow. These requirements motivate the use of a unified component interface, tool-specific adapters, and a framework architecture that is independent of any single modeling formalism or simulation back-end.

The *simulation* requirements define the technical capabilities needed to execute the coupled system model. These include graph-based dependency analysis, execution-order determination, algebraic-loop handling, master-algorithm selection, macro-step coordination, and support for hybrid event handling.

The *interaction* requirements define supporting capabilities for inspecting, validating, and analyzing system models and simulation results. These include structural validation of the assembled system, visualization of the dependency structure, and access to simulation histories for post-processing and comparison of results.

These categories provide the direct link between the user requirements and the framework architecture presented in the following chapter. Individual implementation features are later related back to the corresponding user and system requirements where this is relevant for the discussion.

3.2 Tool Comparison and Selection

The external tools used in this thesis were selected based on the requirements derived in the previous section. The comparison is therefore not intended as a comprehensive survey of all available modeling and simulation environments. Instead, it focuses on those tools that were relevant for the required workflow and that were considered realistic candidates for integration into *SysSimX*. For each modeling domain, the selected tool is justified with respect to the required capabilities, the integration effort, and the overall scope of the thesis.

3.2.1 Equation-Based Modeling

For the equation-based part of the workflow, the main distinction is between signal-based and equation-based modeling paradigms. Signal-based modeling describes systems as directed chains of inputs and outputs. It is well suited for control logic, signal processing, and explicitly causal subsystem descriptions. Equation-based modeling, in contrast, describes the relations between physical quantities declaratively through equations. The computational direction is not fixed in advance, but is determined during model translation. This makes equation-based modeling particularly suitable for multi-domain physical systems with constraints and conservation laws.

Within the considered tools, Simulink represents the signal-based approach, whereas Modelica and Simscape support acausal physical modeling. Simulink is based on directed block diagrams and is therefore natural for controllers, signal-flow architectures, and sampled logic [39]. Simscape extends this environment by adding acausal physical components and physical network semantics. However, Simscape remains embedded in the proprietary MATLAB/Simulink ecosystem and relies on explicit interface blocks when physical networks interact with signal-based control models [40].

Modelica follows a more general equation-based approach. It provides a dedicated language for object-oriented, multi-domain, and acausal modeling of physical systems. In contrast to signal-based block modeling, subsystem behavior is formulated directly in terms of

equations and connector relations [4]. This is well aligned with the requirements of the present thesis, since the workflow must support physically meaningful subsystem models, hierarchical composition, and later integration into a heterogeneous co-simulation framework.

For the present work, Modelica was selected as the modeling formalism for equation-based subsystem models, and OpenModelica was selected as the corresponding open-source toolchain. The decisive reasons were the explicit support for equation-based and acausal modeling, the suitability for multi-domain physical systems, the availability of an open-source compiler, and the possibility to integrate the resulting models into the overall workflow through standard co-simulation interfaces. Simulink was considered relevant as a representative signal-based environment, and Simscape as an acausal physical extension within that environment. However, the thesis required an open and scriptable workflow centered on equation-based physical modeling rather than a proprietary block-diagram environment. For this reason, Modelica/OpenModelica was the more suitable choice.

Table 3.5: Relevant comparison of equation-based modeling candidates.

Tool	Paradigm	Relevant strengths	Main limitation for this thesis
Simulink	Signal-based, causal	Strong for control logic, signal processing, and hierarchical block models	Not centered on equation-based physical modeling
Simscape	Acausal physical modeling within Simulink	Physical network modeling, combination with Simulink control models	Embedded in proprietary MATLAB/Simulink environment
Modelica / OpenModelica	Equation-based, acausal	Multi-domain physical modeling, declarative equations, open-source toolchain, suitable for co-simulation export	Requires external toolchain for translation and execution

3.2.2 Musculoskeletal Modeling

For the musculoskeletal part of the workflow, the selected tool had to support articulated biomechanical models with joints, constraints, actuators, and forward dynamic simulation. In addition, the tool had to be accessible from Python and suitable for integration into a custom co-simulation framework. Since the thesis focuses on heterogeneous system integration rather than on benchmarking musculoskeletal modeling environments, the goal of this subsection is not to provide a comprehensive survey of all available tools, but to justify the selected tool with respect to the required capabilities.

OpenSim was selected for this purpose. It is an open-source software platform for modeling, simulating, and analyzing musculoskeletal systems and dynamic movement. OpenSim provides forward dynamic simulation, a component-based biomechanical model structure, and programmatic access through its application programming interface [5, 14]. These

properties make it suitable for representing biomechanical subsystems within the heterogeneous workflow considered in this thesis.

From the perspective of framework integration, OpenSim offers three decisive advantages. First, it provides an established modeling environment for rigid-body and musculoskeletal dynamics, including joints, constraints, actuators, and contact-related elements. Second, it supports forward simulation of biomechanical models, which is required for coupling the subsystem to controllers and external components. Third, its Python interface enables direct integration into the *SysSimX* architecture without requiring an intermediate proprietary execution environment.

Other musculoskeletal modeling tools exist, including both research-oriented and commercial environments. However, a systematic comparison of such tools was outside the scope of this thesis. The selection of OpenSim was therefore based on its suitability for the required workflow, its open-source availability, and its practical integrability into the Python-based co-simulation framework.

3.2.3 Finite Element Modeling

For the finite element modeling part of the workflow, the selected tool had to support spatially discretized models, programmable definition of material laws and boundary conditions, and a level of flexibility that allows integration into a custom research-oriented co-simulation framework. In addition, the tool had to be accessible from Python, since the overall architecture of *SysSimX* is implemented in Python and relies on backend adapters for coupling heterogeneous subsystem models. As in the musculoskeletal case, the purpose of this subsection is not to provide a comprehensive benchmark of finite element tools, but to justify the selected tool with respect to the requirements of the thesis.

Netgen/NGSolve was selected for this purpose. It is an open-source finite element environment that combines mesh generation with a programmable finite element library [41, 42]. In contrast to solver environments that are primarily organized around graphical pre-processing and fixed analysis workflows, NGSolve provides direct programmatic access to the mathematical problem formulation, including finite element spaces, weak forms, loads, materials, boundary conditions, grid functions, and solver configuration. This makes it well suited for integrating finite element models, such as continuum-mechanical subsystem models, into a custom co-simulation framework.

From the perspective of framework integration, the decisive advantage of Netgen/NGSolve is its Python interface. It allows the finite element model to be assembled, parameterized, advanced, and queried directly from the same programming environment in which the co-simulation framework is implemented. This is particularly useful for the present work, where the finite element subsystem must be wrapped as a co-simulation component and coupled to equation-based and musculoskeletal subsystem models through a common interface.

The selected tool also reflects the scope of the thesis. The goal was to demonstrate the integration of a finite element subsystem into a heterogeneous hybrid simulation workflow,

not to perform an extensive comparative study of finite element platforms. Other open-source tools such as CalculiX [43], FEBio [44], or openCFS [45] were considered at a high level. However, for the present work, their integration paths were assessed as less direct for a Python-centered prototype workflow, for example due to less mature programmatic interfaces or a stronger focus on file-based pre-processing and execution. Netgen/NGSolve was therefore chosen because it combines strong finite element capabilities with direct scriptability and practical suitability for framework integration.

A limitation of this choice is that NGSolve requires a comparatively strong mathematical understanding of finite element modeling, since many aspects of the model must be defined directly in code. This limitation was accepted because the thesis focuses on framework integration and controllable programmatic access rather than on interactive finite element model authoring.

3.2.4 Summary

In summary, the selected toolchain covers the three modeling domains required in this thesis:

- Modelica/OpenModelica for equation-based physical modeling,
- OpenSim for musculoskeletal biomechanics, and
- Netgen/NGSolve for continuum-mechanical finite element modeling.

Each tool was chosen because it satisfies the relevant domain-specific requirements while remaining sufficiently accessible for integration into a Python-based workflow. At the same time, no single existing environment covered the full combination of heterogeneous model integration, hybrid co-simulation, and unified orchestration required for this work. This gap motivated the development of the *SysSimX* framework presented in the following chapters.

Framework Architecture

This chapter presents the architecture of *SysSimX*. It begins with an overview of the layered structure and the design principles that guided its development. The subsequent sections then describe the core abstractions that define the component interface, the system model, the execution algorithms, and the multi-model component concept. Implementation details and verification of individual features are deferred to Chapter 5.

4.1 Architectural Overview

Figure 4.1 shows the layered architecture of *SysSimX* and the delegation chain from user code through the framework to the external simulation backends. The architecture is organized into four internal layers, adapter, core abstractions, system orchestration, and execution, each with a clearly defined responsibility. Six design principles shaped this layering:

1. *component-based decomposition,*
2. *tool-agnostic core with backend adapters,*
3. *separation of model and orchestration,*
4. *pluggable execution algorithms,*
5. *explicit and unit-aware interfaces, and*
6. *progressive capability disclosure.*

Rather than discussing these principles in isolation, the following paragraphs introduce each layer of the architecture and explain the design decisions that motivated it. The description proceeds from the bottom of Figure 4.1 upward, starting with the external tools and ending with the execution algorithms.

External simulation tools. The simulation backends at the bottom of Figure 4.1 are not part of the framework. They include FMI 2.0 Co-Simulation FMUs exported from equation-based modeling environments, OpenSim for musculoskeletal dynamics, and Netgen/NGSolve for finite element analysis. Each tool provides its own solver, model representation, and programmatic Python interface. The framework treats these tools as black-box backends and interacts with them exclusively through their respective APIs.

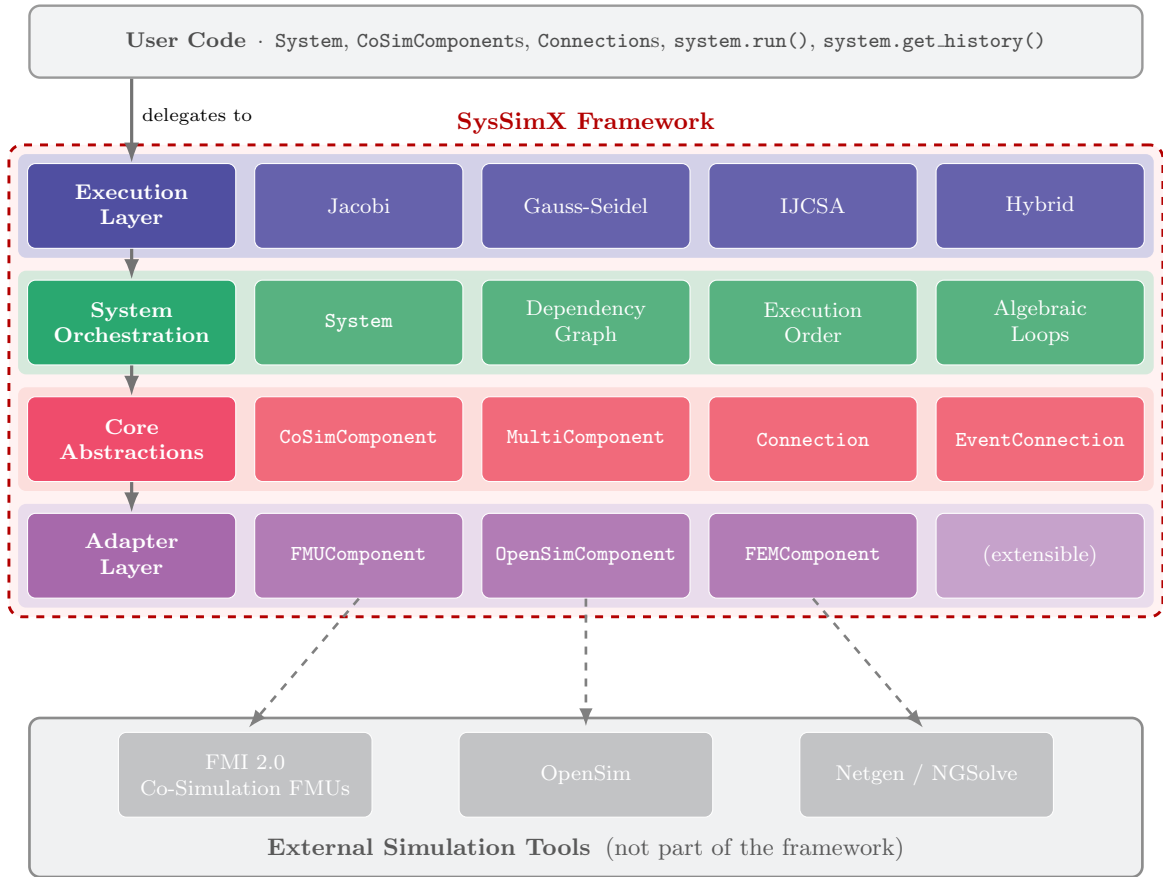


Figure 4.1: Layered architecture of the *SysSimX* framework. Solid arrows indicate the delegation direction between layers; dashed arrows indicate calls to external tool APIs across the framework boundary.

Adapter layer. The adapter layer wraps each tool-specific API in a class that implements the common `CoSimComponent` interface. `FMUComponent` maps FMI function calls to the component lifecycle, while `OpenSimComponent` and `FEMComponent` do the same for OpenSim and NGSolve, respectively. As a result, every subsystem model, regardless of its origin, exposes the same interface for initialization, time stepping, input and output exchange, and state access. This realizes the principle of a *tool-agnostic core with backend adapters*. All tool-specific code is confined to the adapter boundary, and the layers above can treat heterogeneous models uniformly. The adapter layer is designed to be extensible. Additional backends can be integrated by providing a new adapter class without modifying the layers above.

Core abstractions. The core abstraction layer defines the modeling primitives of the framework. The abstract base class `CoSimComponent` specifies the contract that every simulation component must satisfy: typed and unit-annotated input and output ports, a fixed lifecycle for initialization, stepping, and output evaluation, and optional hooks for direct-feedthrough declaration, event indicators, and state rollback. This reflects two design prin-

ciples. First, *component-based decomposition*: a system is represented as a set of interacting components that can be constructed, configured, and exchanged independently. Second, *progressive capability disclosure*: the interface distinguishes between a minimal required contract and optional capabilities that the framework detects through introspection. Simple components remain lightweight, while components that participate in algebraic-loop resolution, hybrid event handling, or multi-model switching can expose the additional information needed by the corresponding algorithms.

Signal connections and event connections define the directed data flow and discrete event propagation between components. *Explicit and unit-aware interfaces* make coupling assumptions visible: each port carries a declared type and physical unit, and connections are validated for compatibility when they are added to the system. The `MultiComponent` class extends this layer to the case where several interchangeable models represent the same physical subsystem, providing a single external interface while managing multiple internal realizations with consistent state synchronization.

System orchestration. The `System` class assembles components and connections into a complete co-simulation model. It validates port compatibility, constructs the dependency graph, derives a topologically sorted execution order, identifies direct-feedthrough dependencies, and detects algebraic loops through strongly connected component analysis. These structural properties are computed automatically during initialization and made available to the execution algorithms. This layer embodies the *separation of model and orchestration*: components are responsible for their own state integration and output evaluation, while the `System` class handles all system-level structural concerns.

Execution layer. The execution layer at the top of the framework contains the master algorithms that advance the coupled system in time. An algorithm receives a `System` instance and steps all components from the current communication time t to $t + \Delta t$, managing input propagation, output collection, and, where required, iterative resolution of algebraic loops or detection, localization, and handling of discrete events. The same system model can be executed with different algorithms without modifying the component implementations or the system definition. This realizes the principle of *pluggable execution algorithms*: the framework delegates all stepping logic to interchangeable algorithm objects, keeping numerical orchestration separate from model structure.

In addition to the main layers, the framework provides supporting services for unit handling, recording of port and component histories during simulation, and graph-based visualization of the system structure.

The following sections describe each of the core abstractions in detail, beginning with the `CoSimComponent` interface.

4.2 Core Abstractions

This section describes the four core abstractions of *SysSimX*: the component interface, the system model, the algorithm interface, and the multi-model component. Together, they define the contracts on which the entire framework operates. Their concrete realization and verification are discussed in Chapter 5.

4.2.1 CoSimComponent Interface

The abstract base class `CoSimComponent` defines the common interface that every simulation component in *SysSimX* must implement. It is the central contract between tool-specific model wrappers and the co-simulation framework, and allows components from different backends to be treated uniformly during initialization, time stepping, data exchange, and result collection (see Figure 4.2).

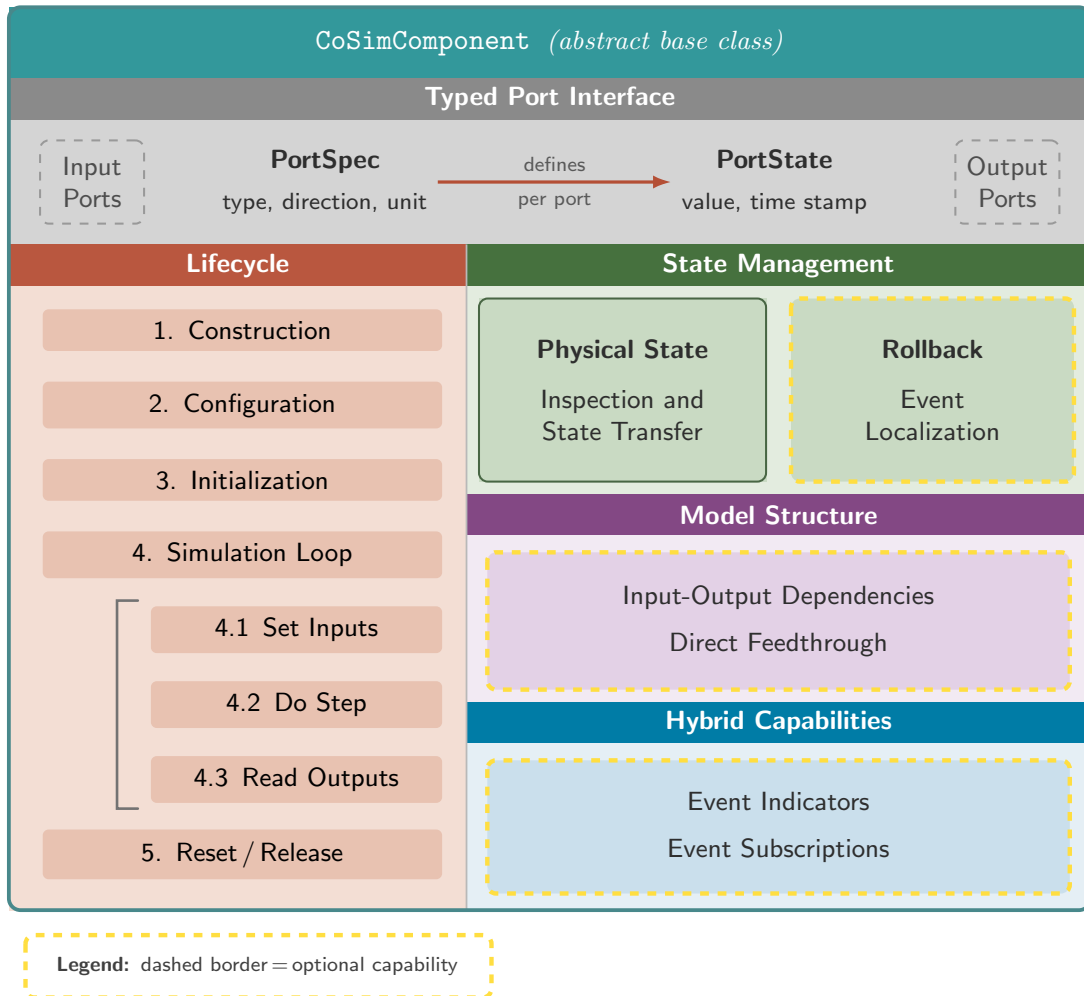


Figure 4.2: Architectural overview of the abstract `CoSimComponent` interface in *SysSimX*. It combines a typed port interface, a uniform lifecycle, distinct mechanisms for physical state transfer and rollback, and optional capabilities for structural analysis and hybrid co-simulation.

A component exposes its system interface through typed input and output ports. Each port carries a declared type (real, integer, boolean, string, or event) and an optional physical unit. This makes coupling assumptions explicit at the interface level and allows the framework to validate type compatibility and unit consistency when connections are established.

The component lifecycle follows a fixed sequence: construction, parameter configuration, initialization at the simulation start time, and then repeated participation in the simulation loop, where inputs are set, one macro step is performed, and outputs are updated and recorded in the history. After the simulation completes, the component is released or reset for a subsequent run. This lifecycle gives the framework a uniform control flow across all component types and is enforced identically regardless of the underlying backend.

The interface separates two forms of state handling. Physical state transfer exports the state in terms of physically meaningful quantities such as positions, velocities, or mode indicators and is used for inspection and for synchronization between alternative models of the same subsystem. Rollback state transfer captures and restores the exact internal solver state of a component and is used during hybrid event handling, where the simulation must return to an earlier time instant and re-advance. This distinction is essential because rollback requires tool-specific solver information that is generally not interpretable across different model representations.

In addition to the required lifecycle methods, the interface defines optional capability hooks for structural analysis and hybrid co-simulation: direct-feedthrough declarations, event indicators, and event subscriptions. The framework uses standardized capability checks and optional registrations to determine which advanced features a component provides. This keeps simple components lightweight while allowing hybrid and strongly coupled components to expose the additional information required by the system-level algorithms.

4.2.2 System and Connections

The **System** class represents a complete co-simulation model. It collects the participating components, stores the connections between them, and provides the structural context required for orchestration. In this way, the definition of individual subsystem models is kept separate from the definition of the coupled system. Figure 4.3 illustrates this separation: the left panel shows a system as defined by the user, while the right panel shows the structural information the framework derives during initialization.

Two types of connections are supported. A signal connection defines a directed data flow from an output port of a source component to an input port of a destination component. An event connection routes a named event from a source component to a handler method in a destination component, enabling cross-component discrete interactions such as mode changes or contact triggers. Both connection types are visible in Figure 4.3 (left): solid arrows denote signal connections, while the dashed arrow represents an event connection. When a connection is added, the **System** validates that the referenced ports exist and that the connected ports are type- and unit-compatible.

From the registered components and connections, the **System** builds a dependency graph

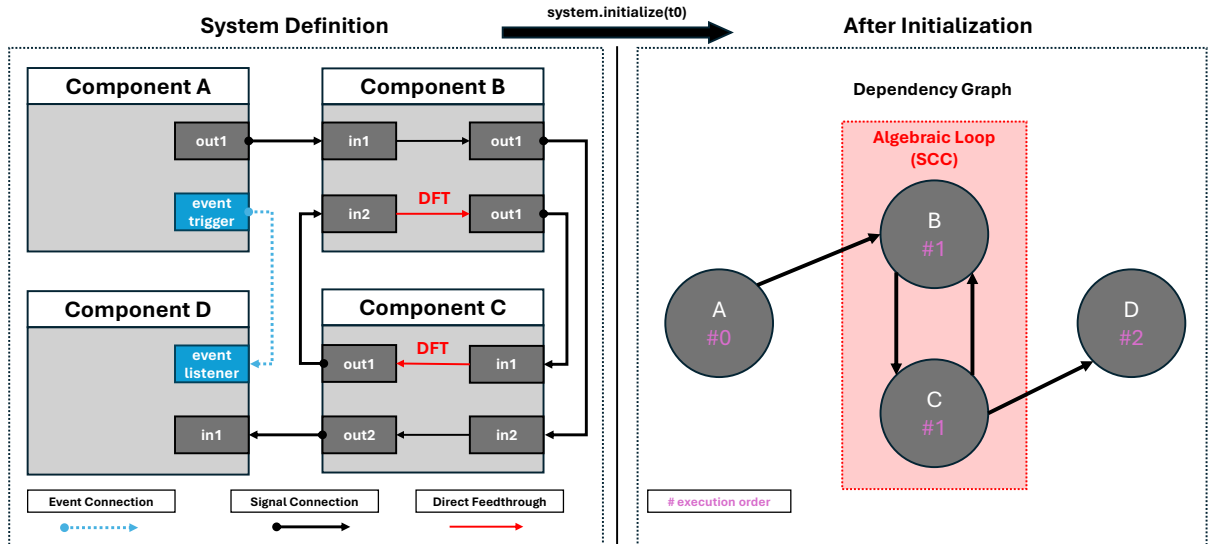


Figure 4.3: System definition and initialization. The left panel shows the user-defined system model with four components, their typed ports, signal connections (solid arrows), an event connection (dashed blue), and direct-feedthrough paths (solid red). The right panel shows the result of `system.initialize(t0)`: the framework derives a dependency graph, computes a topologically sorted execution order ($\#$), and identifies algebraic loops as strongly connected components (shaded region).

in which components form the nodes and directed connections form the edges. This graph is the basis for the structural analysis performed during initialization: the framework derives a topologically sorted execution order, identifies direct-feedthrough dependencies, and detects algebraic loops. As shown in Figure 4.3 (right), components B and C form a strongly connected component due to their mutual direct-feedthrough dependencies, which the framework flags as an algebraic loop. The details of these analyses are discussed in Chapter 5; at the architectural level, it is sufficient to note that they are computed automatically and made available to the execution algorithms.

The `System` class does not implement a stepping scheme itself. Instead, it holds a reference to an `Algorithm` object and delegates the simulation loop to it. When the user calls `system.run()`, the system advances in macro steps, each of which is executed by `algorithm.step(system, t, dt)`. The `System` therefore acts as the central coordination point between model structure and numerical execution, while the stepping policy remains exchangeable.

4.2.3 Algorithm Interface

The execution of a co-simulation step is delegated to objects that implement the abstract base class `Algorithm`. An algorithm advances a given `System` from the current communication time t to $t + \Delta t$ by stepping components, propagating interface values, and—where required—resolving algebraic coupling or handling discrete events. This realizes a strategy-based design in which the system structure is independent of the chosen execution scheme.

SysSimX provides algorithms that range from explicit parallel and sequential schemes to implicit algebraic-loop solvers and hybrid event-driven execution. The choice depends on the

structural and dynamical properties of the system; when event indicators are present in any registered component, the framework automatically upgrades to the hybrid algorithm during initialization. All algorithms share the same system and component interfaces, so the user can change the coordination scheme without modifying the model definitions. The individual algorithms and their numerical procedures are described in Sections ?? through ??.

4.2.4 Multi-Model Component

The class `MultiComponent` extends the `CoSimComponent` abstraction to the case where several interchangeable models represent the same physical subsystem. It provides a single external component interface while managing multiple internal model realizations. This abstraction is the basis for multi-fidelity simulation and runtime model switching in *SysSimX*. Figure 4.4 illustrates the internal structure and the external view of the abstraction.

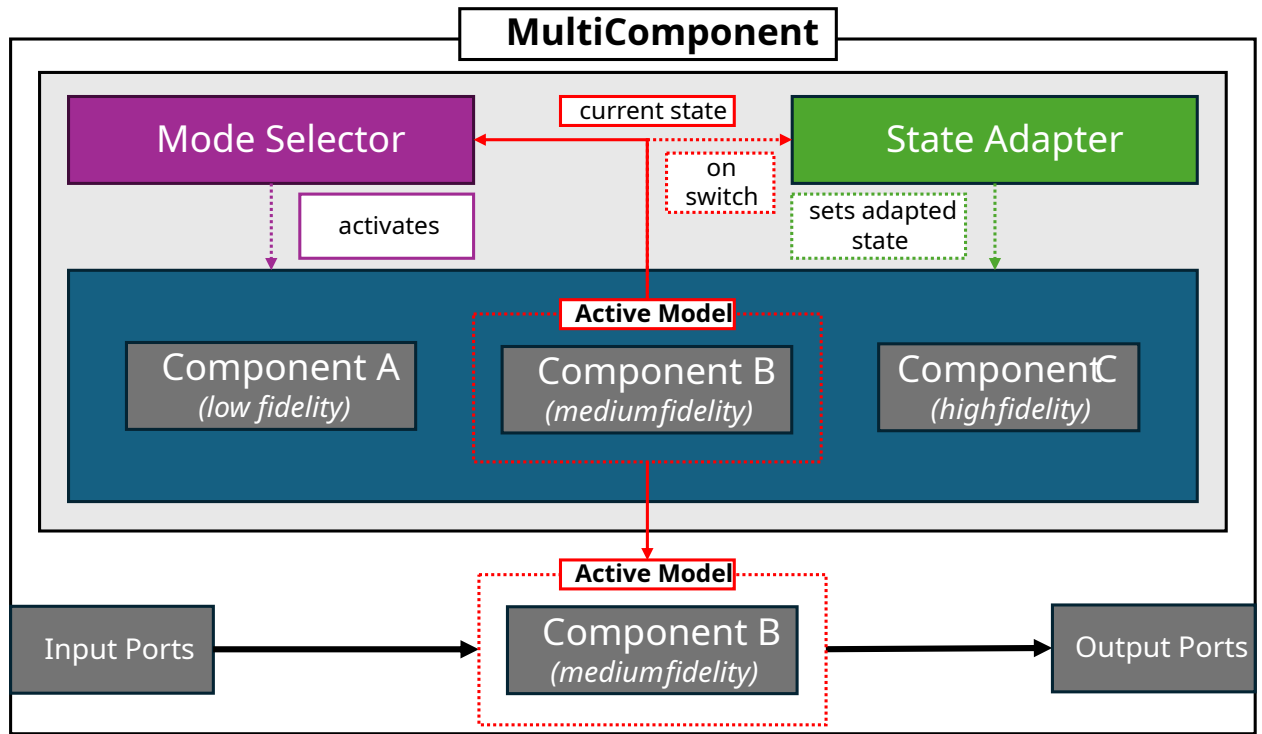


Figure 4.4: Architectural overview of the `MultiComponent` abstraction. The upper panel shows the internal structure: multiple interchangeable models are registered under a single component wrapper. The mode selector determines the active model based on the current state; on a mode switch, the state adapter transfers and adapts the physical state to the target model. The lower panel shows the external view: the surrounding system interacts only with the active model through unified input and output ports.

At any point in time, exactly one internal model is active, as indicated by the highlighted component in Figure 4.4. A user-defined mode selection rule determines when the active model should change based on the current state of the active model. The surrounding system interacts only with the unified external interface, as shown in the lower panel of Figure 4.4, and is therefore independent of the currently selected internal model. This makes it possible

to switch between models, for example between a computationally inexpensive rigid-body model and a high-fidelity finite element model, without altering the system structure.

A mode change requires state synchronization between the outgoing and the incoming model. The `MultiComponent` uses physical state transfer for this purpose, because the transferred quantities must be interpretable by a model with a potentially different internal formulation. If the state representations differ, a user-provided state adaptation function maps the exported state to the format expected by the target model.

To avoid rapid switching near decision boundaries, the abstraction supports hysteresis through a configurable minimum dwell time or switching margin. This prevents chattering and reduces unnecessary switching overhead. The detailed switching mechanism and its integration with the hybrid algorithm are discussed in Section 5.7.

4.3 Summary

Feature Implementation and Verification

The previous chapter introduced the architectural abstractions of *SysSimX*: the component interface, the system model, the algorithm strategy, and the multi-model component. This chapter describes how each of these abstractions is realized in practice. It covers the concrete implementation of the framework’s core features, from the port system that enforces interface contracts to the hybrid algorithm that handles discrete events during co-simulation.

Each section addresses one feature of the framework and follows a common structure. It begins with a brief reference to the architectural or requirements context that motivates the feature, then describes the implementation approach and the relevant technical details, and concludes with a focused verification example that demonstrates the feature in isolation. The full integration of all features is deferred to the case study in Chapter 6.

The sections are ordered from foundational to advanced. Section 5.1 describes the typed, unit-aware port system that underlies all data exchange. Section 5.2 presents the three backend-specific component wrappers that map external simulation tools to the common `CoSimComponent` interface. Section 5.3 covers the graph-based structural analysis that derives execution order and detects algebraic loops from the system topology. Section 5.4 describes the iterative resolution of algebraic loops identified during structural analysis. Section 5.5 introduces the master algorithms that advance the coupled system in time, building on the execution order and loop resolution from the preceding sections. Section 5.6 extends the stepping logic with event detection, localization, and rollback for hybrid co-simulation. Section 5.7 describes the multi-model switching mechanism with state adaptation and hysteresis. Finally, Section 5.8 presents the graph-based visualization of the system structure.

5.1 Port System and Unit-Aware Connections

The need for a port system arises from the component-based modeling approach of system models where component interfaces are defined by input and output ports. A system model is created by defining connections between output ports of source and input ports of destination components.

The need for a port system and unit aware connections is derived from the user requirement UR_01 and UR_02 to enable the component-based modeling of subsystems and the explicit use of physical units at model interfaces. Further, the architectural overview in

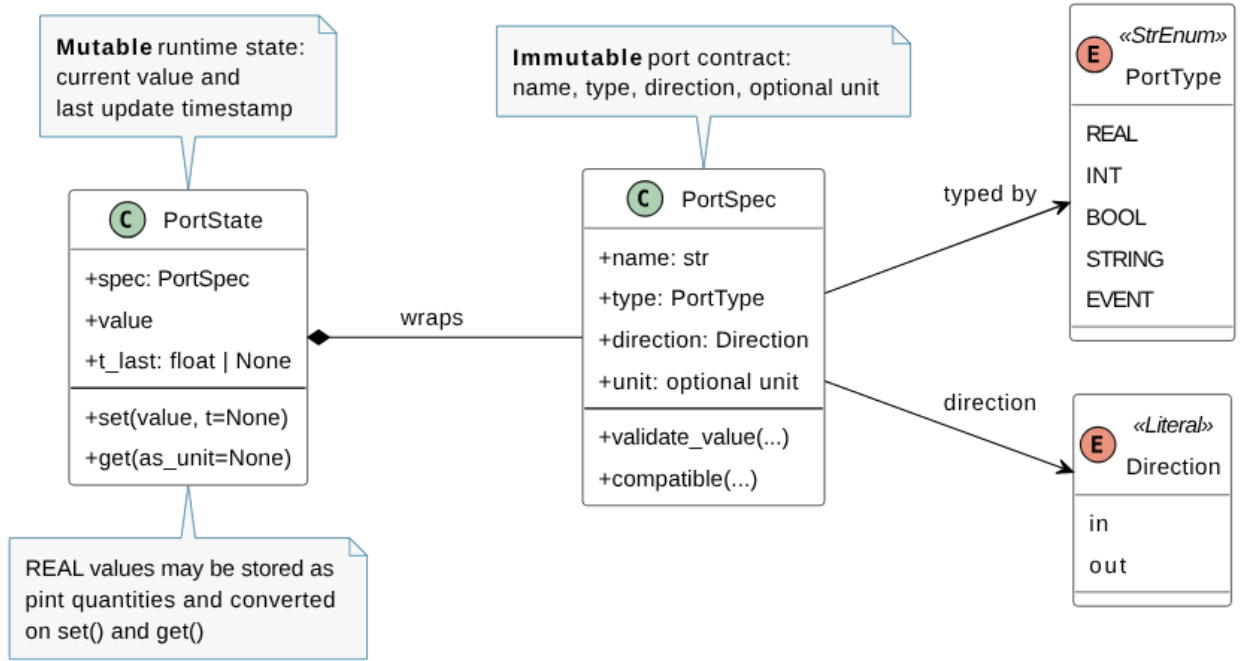


Figure 5.1: Enter Caption

Section 4.2.1 introduced typed and unit-annotated ports as the mechanism through which components expose their interfaces. This section describes the concrete realization of the port system, including the separation of interface definition from runtime state, the integration of physical unit handling, and the validation logic that enforces interface compatibility at connection time.

5.1.1 Port Specification and Port State

The port system separates two concerns into distinct data structures. A **PortSpec** defines the static interface contract of a single port by its name, direction (input or output), data type, and optional physical unit. The port spec is setup during the component construction. Five data types are supported through the **PortType** enumeration: **REAL**, **INTEGER**, **BOOLEAN**, **STRING**, and **EVENT**. Only **REAL** ports carry a physical unit.

A **PortState** holds the mutable runtime value of a port during simulation. Each **PortState** references the **PortSpec** that defines its contract and additionally stores the current value and a timestamp of the last update. When a value is assigned through `set()`, the port state validates the value against its specification and, for **REAL** ports, performs automatic unit conversion to the port's declared unit. When a value is retrieved through `get()`, an optional target unit can be specified, and the port state converts on the fly. Listing ?? illustrates this separation.

This design ensures that port contracts are defined once and remain stable throughout the simulation, while the runtime values can change at every communication step. Components developed independently can be connected as long as their port specifications are compatible, regardless of whether they internally use different unit conventions.

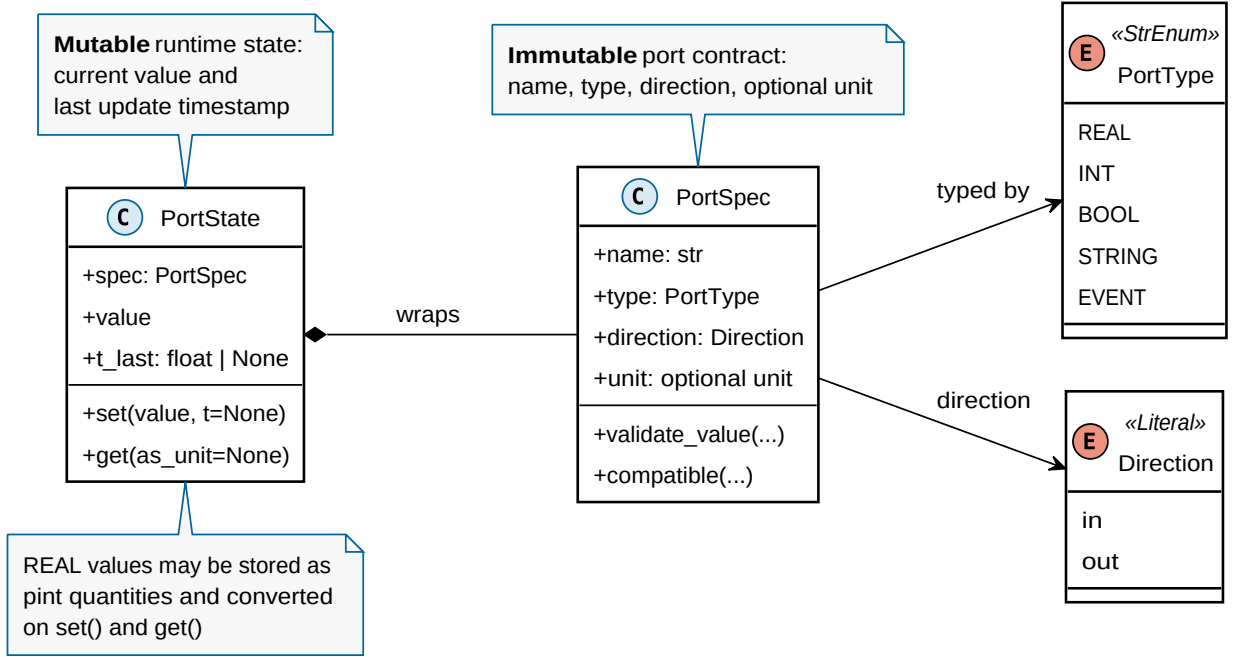


Figure 5.2: Enter Caption

5.1.2 Physical Unit Handling

Physical units are managed through the Pint library [pint], which provides a unit registry, dimensional analysis, and automatic conversion between compatible units. The framework initializes a shared unit registry with case-sensitive parsing and automatic offset-to-base-unit conversion, the latter being relevant for temperature units that involve an additive offset.

Unit enforcement occurs at three stages of the simulation workflow. First, when a `PortSpec` is constructed, the declared unit string is validated against the Pint registry. Unrecognized or malformed unit strings are rejected immediately, preventing silent errors during later simulation phases. Second, when a connection between two ports is added to the system, the framework checks dimensional compatibility between the source and destination units, as described in Section 5.1.3. Third, at runtime, every call to `PortState.set()` converts the assigned value to the port's declared unit. If the value carries an incompatible dimension—for example, assigning a force quantity to a port declared with a length unit—a `DimensionalityError` is raised.

To support equation-based models exported through the Functional Mock-up Interface, the unit registry includes a preprocessing step that normalizes FMU-style unit strings. The FMI specification uses a compact notation where exponents are written without a caret (e.g., "m.s-2" for m s^{-2}), which is not directly recognized by Pint. The framework translates such strings into the standard form (e.g., "m*s**-2") before passing them to the registry.

5.1.3 Connection Validation

When the user adds a connection to the `System`, the framework performs a sequence of validation checks before accepting it. First, it verifies that both the source and destination components are registered in the system. Second, it checks that the named output port exists on the source component and the named input port exists on the destination component. Third, it enforces type compatibility: the `PortType` of the source must match that of the destination. A `REAL` output cannot be connected to a `BOOLEAN` input, even if the underlying numerical representations would permit it.

For `REAL` ports, a fourth check verifies unit compatibility through the static method `PortSpec.compatible()`. Three cases are distinguished:

- If both ports declare no unit, the connection is accepted as dimensionless.
- If one port declares a unit and the other does not, the connection is *rejected*. This asymmetry is treated as an error rather than a default assumption, because silently connecting a dimensioned quantity to a dimensionless port is a common source of modeling mistakes.
- If both ports declare a unit, the framework checks whether the two units are dimensionally equivalent using `Pint`. Units that differ only by a scaling factor (e.g., `N` and `kN`, or `rad` and `deg`) are accepted; units with incompatible dimensions (e.g., `N` and `m`) are rejected.

All validation checks are performed when the connection is added, not during simulation. This fail-early design ensures that structural modeling errors—such as wiring a force output to a displacement input—are detected during system assembly, before any numerical computation begins.

5.1.4 Verification

The port system is verified through unit tests that cover three categories: type safety, unit compatibility, and runtime conversion.

Type safety is tested by attempting to assign values of incorrect types to ports. Assigning a string to a `REAL` port, or a float to a `BOOLEAN` port, must raise a validation error.

Unit compatibility is tested by constructing pairs of `PortSpec` objects with various unit combinations and checking the result of `PortSpec.compatible()`. Table 5.1 summarizes the expected outcomes.

Runtime conversion is tested by setting a value on a port in one unit and retrieving it in another. For example, setting a torque port to 5.0 N m and retrieving with `get(as_unit="N*cm")` must return 500.0 N cm. These tests confirm that the unit conversion pipeline—from assignment through internal storage to retrieval—preserves physical consistency without requiring the user to perform manual conversions.

Table 5.1: Expected outcomes of port compatibility checks for **REAL** ports with different unit declarations.

Source unit	Destination unit	Result
N*m	N*m	Compatible
N	kN	Compatible (same dimension)
rad	deg	Compatible (same dimension)
N	m	Rejected (dimension mismatch)
N*m	(none)	Rejected (asymmetric units)
(none)	(none)	Compatible (dimensionless)

5.2 Component Wrappers

5.2.1 FMI 2.0 Co-Simulation Wrapper

5.2.2 Musculoskeletal Model Wrapper

5.2.3 Finite Element Model Wrapper

5.3 Structural Analysis and Execution Ordering

5.4 Algebraic Loop Resolution

5.5 Master Algorithms

5.6 Hybrid Co-Simulation Algorithm

5.7 Multi-Model Component and Mode Switching

5.8 System Graph Visualization

CHAPTER 6

Case Study: Controlled Pendulum System

6.1 Choice and Justification of the Case Study

6.2 System Engineering of the Controlled Pendulum

6.2.1 System Architecture and Signal Flow

6.2.2 PID Controller

6.2.3 BLDC Drive Model

6.2.4 Sensors and Quantization

6.3 Pendulum Models

6.3.1 Modelica Pendulum

6.3.2 OpenSim Pendulum

6.3.3 FEM Pendulum

1. Mesh generation and geometry
2. Neo-Hookean Material Law
3. Variational Formulation
4. Newmark Time Integration Scheme
5. Torque Application
6. Contact Boundary
7. Rigid Body Mappings

6.3.4 Multi-Model Pendulum

6.4 System Assembly and Configuration

6.4.1 Component Wiring and Connection Graph

6.4.2 Algebraic Loop Analysis

6.4.3 Monolithic Modelica Reference Model

CHAPTER 7

Discussion and Conclusion

7.1 Discussion of Results

7.2 Limitations

7.3 Conclusions

7.4 Outlook and Future Work

List of Figures

4.1	Layered architecture of the <i>SysSimX</i> framework. Solid arrows indicate the delegation direction between layers; dashed arrows indicate calls to external tool APIs across the framework boundary.	25
4.2	Architectural overview of the abstract CoSimComponent interface in <i>SysSimX</i> . It combines a typed port interface, a uniform lifecycle, distinct mechanisms for physical state transfer and rollback, and optional capabilities for structural analysis and hybrid co-simulation.	27
4.3	System definition and initialization. The left panel shows the user-defined system model with four components, their typed ports, signal connections (solid arrows), an event connection (dashed blue), and direct-feedthrough paths (solid red). The right panel shows the result of <code>system.initialize(t0)</code> : the framework derives a dependency graph, computes a topologically sorted execution order (<code>#</code>), and identifies algebraic loops as strongly connected components (shaded region).	29
4.4	Architectural overview of the MultiComponent abstraction. The upper panel shows the internal structure: multiple interchangeable models are registered under a single component wrapper. The mode selector determines the active model based on the current state; on a mode switch, the state adapter transfers and adapts the physical state to the target model. The lower panel shows the external view: the surrounding system interacts only with the active model through unified input and output ports.	30
5.1	Enter Caption	33
5.2	Enter Caption	34

List of Tables

3.5	Relevant comparison of equation-based modeling candidates.	21
5.1	Expected outcomes of port compatibility checks for <code>REAL</code> ports with different unit declarations.	36

List of Listings

Bibliography

- [1] E. Lee, *Cyber Physical Systems: Design Challenges*, Journal Abbreviation: Electrical Engineering and Computer Sciences Pages: 369 Publication Title: Electrical Engineering and Computer Sciences, **2008**.
- [2] H. Van der Auweraer, J. Anthonis, S. De Bruyne, J. Leuridan, “Virtual engineering at work: The challenges for designing mechatronic products”, *ResearchGate*, DOI 10.1007/s00366-012-0286-6.
- [3] R. Eberle, M. Ebenbichler, B. Reimeir, L. Ralfs, R. Weidner, “Biomechanical Simulation Pipeline for Exoskeletons - A digital tool for the targeted development of support systems”, *Industry 4.0 Science* **2025**, 2025, DOI 10.30844/I4SE.25.5.30.
- [4] P. Fritzson, *Principles of object oriented modeling and simulation with Modelica 3.3: a cyber-physical approach*, 2nd edition, John Wiley & Sons Inc, Hoboken, New Jersey, **2015**.
- [5] S. L. Delp, F. C. Anderson, A. S. Arnold, P. Loan, A. Habib, C. T. John, E. Guendelman, D. G. Thelen, “OpenSim: Open-Source Software to Create and Analyze Dynamic Simulations of Movement”, *IEEE Transactions on Biomedical Engineering* **2007**, 54, 1940–1950.
- [6] C. Gomes, C. Thule, D. Broman, P. G. Larsen, H. Vangheluwe, “Co-Simulation: State of the Art”, *ACM Computing Surveys* **2019**, 51, 1–33.
- [7] Functional Mock-up Interface Specification 2.0.
- [8] D. Broman, L. Greenberg, E. A. Lee, M. Masin, S. Tripakis, M. Wetter in Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control, ACM, Seattle Washington, **2015**, pp. 179–188.
- [9] S. Sicklinger, V. Belsky, B. Engelmann, H. Elmqvist, H. Olsson, R. Wüchner, K.-U. Bletzinger, “Interface Jacobian-based Co-Simulation”, *International Journal for Numerical Methods in Engineering* **2014**, 98, 418–444.
- [10] D. Broman, C. Brooks, L. Greenberg, E. A. Lee, M. Masin, S. Tripakis, M. Wetter in 2013 Proceedings of the International Conference on Embedded Software (EMSOFT), IEEE, Montreal, QC, Canada, **2013**, pp. 1–12.

-
- [11] F. Cremona, M. Lohstroh, D. Broman, M. Di Natale, E. A. Lee, S. Tripakis in 2016 ACM/IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE), **2016**, pp. 173–183.
 - [12] F. Cremona, M. Lohstroh, D. Broman, E. A. Lee, M. Masin, S. Tripakis, “Hybrid co-simulation: it’s about time”, *Software & Systems Modeling* **2019**, *18*, 1655–1679.
 - [13] S. H. Choi, S. J. Lee, T. G. Kim, Multi-fidelity modeling & simulation methodology for simulation speed up | Proceedings of the 2nd ACM SIGSIM Conference on Principles of Advanced Discrete Simulation, en, **2014**.
 - [14] A. Seth, J. L. Hicks, T. K. Uchida, A. Habib, C. L. Dembia, J. J. Dunne, C. F. Ong, M. S. DeMers, A. Rajagopal, M. Millard, S. R. Hamner, E. M. Arnold, J. R. Yong, S. K. Lakshmikanth, M. A. Sherman, J. P. Ku, S. L. Delp, “OpenSim: Simulating musculoskeletal dynamics and neuromuscular control to study human and animal movement”, *PLOS Computational Biology* **2018**, *14*, e1006223.
 - [15] K. Knothe, H. Wessels, *Finite Elemente*, Springer Berlin Heidelberg, Berlin, Heidelberg, **2017**.
 - [16] Functional Mock-up Interface Specification 3.0.2.
 - [17] OMSimulator - Integrated FMI and TLM-based Co-simulation with Composite Model Editing and SSP, en.
 - [18] C. Thule, K. Lausdahl, C. Gomes, G. Meisl, P. G. Larsen, “Maestro: The INTO-CPS co-simulation framework”, *Simulation Modelling Practice and Theory* **2019**, *92*, 45–61.
 - [19] Home - FMPy.
 - [20] C. Andersson, J. Åkesson, C. Führer, “PyFMI: A Python Package for Simulation of Coupled Dynamic Models with the Functional Mock-up Interface”.
 - [21] C. Friedrich, A. Lombana, J. Fasquel, C. Schlick, N. Bennani, M. Mendil, “CoFmuPy: A Python Framework for Rapid Prototyping of FMI-based Digital Twins”.
 - [22] *IEEE recommended practice for validation of computational electromagnetics computer modeling and simulations*, (Eds.: Institute of Electrical and Electronics Engineers, IEEE-SA Standards Board), Institute of Electrical and Electronics Engineers, New York, **2011**.
 - [23] B. Peherstorfer, K. Willcox, M. Gunzburger, “Survey of Multifidelity Methods in Uncertainty Propagation, Inference, and Optimization”, *SIAM Review* **2018**, *60*, 550–591.
 - [24] M. G. Fernández-Godino, “Review of multi-fidelity models”, *Advances in Computational Science and Engineering* **2023**, *1*, 351–400.
 - [25] M. A. Williams, A. G. Alleyne in 2014 American Control Conference, ISSN: 2378-5861, **2014**, pp. 3104–3109.

-
- [26] S. H. Choi, K.-M. Seo, T. G. Kim, “Accelerated Simulation of Discrete Event Dynamic Systems via a Multi-Fidelity Modeling Framework”, *Applied Sciences* **2017**, 7, DOI 10.3390/app7101056.
 - [27] K. J. Åström, R. M. Murray, *Feedback systems: an introduction for scientists and engineers*, Princeton University Press, Princeton, **2008**.
 - [28] D. Karnopp, D. L. Margolis, R. C. Rosenberg, *System dynamics: modeling, simulation, and control of mechatronic systems*, 5. ed, Wiley, Hoboken, NJ, **2012**.
 - [29] P. Kunkel, V. Mehrmann, *Differential-Algebraic Equations: Analysis and Numerical Solution*, 2nd ed., EMS Press, **2024**.
 - [30] F. E. Cellier, E. Kofman, *Continuous system simulation*, Springer, New York, **2006**.
 - [31] F. Cellier, “Combined continuous/discrete simulation: applications, techniques and tools”, **1986**, 24–33.
 - [32] P. Derler, E. A. Lee, A. Sangiovanni Vincentelli, “Modeling Cyber–Physical Systems”, *Proceedings of the IEEE* **2012**, 100, 13–28.
 - [33] M. Andersson in **1990**.
 - [34] M. Andersson, Doctoral Thesis (monograph), Department of Automatic Control, **1994**.
 - [35] A. Van Der Schaft, H. Schumacher, *An introduction to hybrid dynamical systems*, Springer London, London, **2000**.
 - [36] M. Branicky, *General Hybrid Dynamical Systems: Modeling, Analysis, and Control*. Pages: 200, **1995**.
 - [37] M. Branicky, S. Mattsson in **2006**, pp. 31–56.
 - [38] A. Urquía Moraleda, C. Martín Villalba, *Modeling and simulation in engineering using Modelica*, Universidad Nacional de Educación a Distancia (España). Editorial, **2018**.
 - [39] “Simulink Getting Started Guide”.
 - [40] “Simscape Getting Started Guide”.
 - [41] J. Schoberl, “C++11 Implementation of Finite Elements in NGSolve”.
 - [42] J. Schöberl, “NETGEN An advancing front 2D/3D-mesh generator based on abstract rules”, *Computing and Visualization in Science* **1997**, 1, 41–52.
 - [43] CALCULIX: A Three-Dimensional Structural Finite Element Program.
 - [44] S. A. Maas, B. J. Ellis, G. A. Ateshian, J. A. Weiss, “FEBio: Finite Elements for Biomechanics”, *Journal of Biomechanical Engineering* **2012**, 134, 11005–NaN.
 - [45] S. Schoder, K. Roppert, openCFS: Open Source Finite Element Software for Coupled Field Simulation – Part Acoustics, arXiv:2207.04443 [math], **2025**.

.5 Complete Derived System Requirement Specification

This appendix contains the complete set of derived system requirements for the *SysSimX* framework. These requirements were derived from the functional user requirements and specify the technical capabilities that the framework must provide. In the main text, only a compact overview is given. The detailed tables are included here for completeness and traceability.

.5.1 Modeling

U-ID	SR-ID	System Requirement	Rationale	Priority
UR_01	SR_01_01	The framework shall represent subsystem models as reusable components with explicit interfaces.	A common component abstraction is required to model subsystems from different physical domains in a uniform way.	Shall
UR_01	SR_01_02	The framework shall allow components from different physical domains to be combined within one system model.	Multi-domain systems require a common system representation for jointly modeling interacting subsystems.	Shall
UR_02	SR_02_01	The framework shall associate physical units with component parameters and interface variables.	Explicit unit information is required to make physical interfaces unambiguous.	Shall
UR_02	SR_02_02	The framework shall validate unit compatibility when connecting interface variables or assigning parameter values.	Unit validation reduces interface errors and improves physical consistency.	Shall
UR_02	SR_02_03	The framework shall support automatic conversion between compatible units where required.	Compatible subsystems may use different unit conventions while still describing the same physical quantity.	Shall
UR_03	SR_03_01	The framework shall support the assembly of complete system models from interconnected components.	A system-level model must be constructible from reusable subsystem models.	Shall
UR_03	SR_03_02	The framework shall provide explicit signal and event connections between subsystem interfaces.	Explicit connections are required to define information flow and event propagation between components.	Shall
UR_03	SR_03_03	The framework shall allow components to be grouped and labeled to reflect subsystem structure.	Structural grouping improves model organization and supports hierarchical interpretation of the system.	Shall

U-ID	SR-ID	System Requirement	Rationale	Priority
UR_04	SR_04_01	The framework shall provide a common component abstraction for integrating executable equation-based subsystem models.	Equation-based models are part of the target workflow, but their authoring and internal processing may remain external to the framework.	Shall
UR_04	SR_04_02	The framework shall expose the parameters, interface variables, and relevant state access of integrated equation-based models through the common component abstraction.	Equation-based models must participate in the coupled workflow through the same interface as other subsystem models.	Shall
UR_05	SR_05_01	The framework shall provide a common component abstraction for integrating executable musculoskeletal subsystem models.	Musculoskeletal models form an essential subsystem class in the target application domain.	Shall
UR_05	SR_05_02	The framework shall support the exchange of kinematic, dynamic, and parameter information between musculoskeletal models and the rest of the system through explicit interfaces.	Biomechanical subsystems must interact with controllers, sensors, and mechanical subsystems through well-defined interface quantities.	Shall
UR_06	SR_06_01	The framework shall provide a common component abstraction for integrating executable continuum-mechanical subsystem models with spatial discretization.	Continuum-mechanical models are required for deformable subsystems and must be integrated into the same workflow as other subsystem classes.	Shall
UR_06	SR_06_02	The framework shall support the exchange of reduced interface quantities, configuration data, and boundary-related inputs for integrated continuum-mechanical models.	Spatially discretized models typically interact with the rest of the system through reduced interface variables rather than full field data.	Shall
UR_07	SR_07_01	The framework shall support components with continuous-time, discrete-time, and hybrid behavior behind the same component abstraction.	The target workflow includes physical subsystems, sampled controllers, and event-driven behavior.	Shall
UR_07	SR_07_02	The framework shall allow hybrid components to expose event-related information required for discrete state changes and re-initialization.	Hybrid subsystem models require explicit support for events, switching, and state updates.	Shall
UR_08	SR_08_01	The framework shall support parameterized instantiation of components and subsystems.	Parameterization is required to reuse the same model structure in different configurations.	Shall
UR_08	SR_08_02	The framework shall allow multiple instances of the same component type with independent parameter settings.	Independent instances are needed for reusable modeling and comparative studies.	Shall

U-ID	SR-ID	System Requirement	Rationale	Priority
UR_08	SR_08_03	The framework shall support reusable higher-level component abstractions for encapsulating subsystem variants or internal model alternatives.	Reuse at subsystem level reduces model duplication and supports structured model management.	Shall

.5.2 Heterogeneity

U-ID	SR-ID	System Requirement	Rationale	Priority
UR_09	SR_09_01	The framework shall support the integration of executable subsystem models through standardized interfaces where available.	Standardized interfaces reduce integration effort and improve interoperability across tools.	Shall
UR_09	SR_09_02	The framework shall provide backend-specific adapter components for subsystem models that do not expose a standardized co-simulation interface.	Not all relevant domain-specific tools provide a common integration standard.	Shall
UR_09	SR_09_03	The framework shall expose a uniform lifecycle and interface contract for all integrated subsystem models, independent of their backend.	Heterogeneous components must be coordinated through one common framework abstraction.	Shall
UR_09	SR_09_04	The framework shall allow tool-specific parameters and execution settings to be configured through the corresponding adapter components.	Backend-specific configuration is required for correct initialization and execution of external models.	Shall
UR_10	SR_10_01	The framework shall provide a unified component abstraction that can represent equation-based, musculoskeletal, and continuum-mechanical subsystem models within the same system model.	The target workflow combines several domain-specific subsystem classes that must be treated consistently.	Shall
UR_10	SR_10_02	The framework shall support common signal and event connection mechanisms across heterogeneous subsystem models.	Heterogeneous components must exchange data and events through a common interaction model.	Shall
UR_10	SR_10_03	The framework shall include heterogeneous subsystem models in the same dependency analysis and execution-order computation, independent of their backend origin.	System-level orchestration must operate on the coupled structure as a whole, not on isolated backend-specific subsets.	Shall

.5.3 Simulation

U-ID	SR-ID	System Requirement	Rationale	Priority
UR_11	SR_11_01	The framework shall provide a system-level macro-step simulation loop for advancing coupled heterogeneous subsystem models in time.	Coupled subsystem models must be coordinated through one common simulation process.	Shall
UR_11	SR_11_02	The framework shall build a dependency graph from the registered components and their connections.	Graph-based structural information is required for consistent co-simulation orchestration.	Shall
UR_11	SR_11_03	The framework shall determine a valid execution order from the dependency structure of the coupled system.	Execution ordering is required to respect data dependencies between subsystems.	Shall
UR_11	SR_11_04	The framework shall detect algebraic loops in the coupled system structure and provide an iterative method for their numerical treatment.	Strongly coupled interface dependencies require special numerical handling.	Shall
UR_11	SR_11_05	The framework shall support hybrid components that expose event indicators, event-triggered state changes, and event propagation.	Hybrid co-simulation requires explicit support for events at subsystem and system level.	Shall
UR_11	SR_11_06	The framework shall support event localization and rollback for hybrid co-simulation when required by the participating components.	Accurate handling of state events requires restoring component states and resolving event times.	Shall
UR_12	SR_12_01	The framework shall allow users to select the system-level co-simulation master algorithm.	Different coupled systems require different orchestration schemes.	Shall
UR_12	SR_12_02	The framework shall allow users to configure the macro step size used for system-level coordination.	Macro-step size directly affects numerical behavior and runtime of the co-simulation.	Shall
UR_12	SR_12_03	The framework shall forward tool-specific simulation settings to the corresponding subsystem wrappers before or during initialization.	External subsystem models may require backend-specific settings in addition to the system-level configuration.	Shall
UR_12	SR_12_04	The framework shall automatically activate hybrid event handling when event-capable components are present in the system.	Hybrid-specific execution support should be enabled whenever the coupled system requires it.	Shall

.5.4 Interaction

U-ID	SR-ID	System Requirement	Rationale	Priority
UR_13	SR_13_01	The framework should generate a directed representation of the coupled system structure from components and their connections.	A structural graph helps users inspect dependencies and data flow.	Should

U-ID	SR-ID	System Requirement	Rationale	Priority
UR_13	SR_13_02	The framework should visualize component groups, ports, signal connections, and event connections in the system graph.	A detailed graph view improves the interpretability of heterogeneous system models.	Should
UR_13	SR_13_03	The framework should support exporting the generated system graph to external file formats.	Exported figures are useful for documentation, debugging, and reporting.	Should
UR_14	SR_14_01	The framework should provide logging output for simulation progress and relevant execution events.	Execution feedback supports debugging and inspection of simulation behavior.	Should
UR_14	SR_14_02	The framework should record component-level and system-level simulation histories during execution.	Stored histories are required for later inspection of dynamic behavior.	Should
UR_14	SR_14_03	The framework should allow users to retrieve selected histories in structured programmatic form, including optional unit conversion.	Selective access to simulation data improves analysis workflows and reduces unnecessary post-processing effort.	Should
UR_15	SR_15_01	The framework should provide result access in forms that support comparison across different model variants and simulation configurations.	Structured result access is required for repeatable comparison of simulation outcomes.	Should
UR_15	SR_15_02	The framework should preserve component, port, and unit context together with recorded simulation data.	Comparative evaluation requires that recorded results remain interpretable after the simulation run.	Should
UR_16	SR_16_01	The framework should support exporting simulation results to common external data formats.	External export is required for documentation and further analysis outside the framework.	Should
UR_16	SR_16_02	The framework should support post-processing access to simulation results through dictionaries, arrays, or file-based exports.	Post-processing workflows require machine-readable access to recorded results.	Should